



Cấu trúc dữ liệu & giải thuật

Data Structure and Algorithms

Chương 6: SẮP XẾP VÀ TÌM KIẾM

TS. Võ Văn Tuấn Dũng

Trường Đại học Tài nguyên Môi trường TP. HCM

6.1. SẮP XẾP

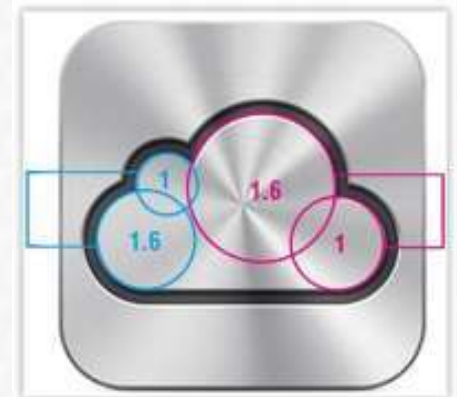
- Sắp xếp là tổ chức lại 1 danh sách các đối tượng theo một thứ tự nhất định.

Ví dụ:

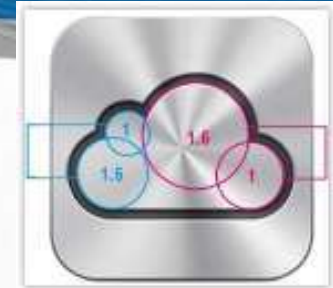
{1, 3, 5, 8, 9, 12}, {"Anh" "Cúc" "Hoàng"}: **tăng dần**

{17, 12, 7, 4, 2, 1}: **giảm dần**

- Việc sắp xếp là một bài toán phổ biến trong tin học.
 - ✓ Do các yêu cầu tìm kiếm thuận lợi.
 - ✓ Sắp xếp kết xuất cho các bảng biểu.
 - ✓ ...



6.1. SẮP XẾP



■ Các bài toán sắp xếp chia thành 2 lớp:

✓ **Sắp xếp trong (*Internal Sorting*):**

- Thực hiện trên danh sách được lưu ở bộ nhớ trong dưới dạng mảng.
- Ưu điểm: giải thuật dễ hiểu và thời gian sắp xếp nhanh.

✓ **Sắp xếp ngoài (*External Sorting*):**

Khi số các đối tượng trong danh sách cần sắp xếp lớn, không thể lưu ở bộ nhớ trong $\Rightarrow$ lưu ở bộ nhớ ngoài dưới dạng các tập tin, mỗi lần sắp thứ tự ta chỉ đưa một phần của dữ liệu vào bộ nhớ để thực hiện $\Rightarrow$ thời gian sắp xếp chậm.

6.1. SẮP XẾP

- Dữ liệu của danh sách cần sắp xếp thường tổ chức thành **mảng các mẫu tin** dữ liệu.
 - ✓ Mỗi mẫu tin thường có một số các **trường dữ liệu** khác nhau.
 - ✓ Trường tham gia quá trình sắp xếp gọi là **khoá (key)**.
 - ✓ Việc sắp xếp sẽ được tiến hành dựa vào **giá trị khoá** này.
- Trong chương này ta chỉ phân tích các phương pháp **sắp xếp trong**.

6.1. SẮP XẾP

■ Hai thuật toán cơ bản mà thuật toán sắp xếp thường sử dụng:

✓ **Đổi chỗ** (swap): thời gian thực hiện là $O(1)$.

```
void swap(datatype &a, datatype &b)
```

```
{  datatype temp = a; // datatype là kiểu dữ liệu của phần tử  
    a = b;  
    b = temp;  
}
```

✓ **So sánh**: Hàm `compare(a, b)` trả lại `TRUE` nếu `a` đi trước `b` trong thứ tự cần sắp xếp và `FALSE` nếu ngược lại.

6.1. SẮP XẾP

■ Các phương pháp sắp xếp

1. Selection Sort (*)
2. Insertion Sort (*)
3. Bubble Sort (*)
4. Interchange Sort
5. Shell sort
6. Quick sort (*)
7. Radix...



6.1. SẮP XẾP

-  **Mô tả bài toán:** Để tiện cho việc minh họa các thuật toán sắp xếp ta mô tả bài toán như sau:
- Cho một mảng các phần tử **e**, mỗi phần tử trong mảng có một thuộc tính khóa. Hãy sắp xếp tăng hoặc giảm các phần tử trong mảng theo giá trị khóa này.
 - Do mỗi phần tử có giá trị khóa nên ta gọi **k[1..n]** là mảng các khóa của các phần tử trong **e**.
 - Yêu cầu: sắp xếp các giá trị này sao cho mảng **k** có thứ tự tăng.

6.1.1. Selection Sort (sắp xếp lựa chọn)

1. Ý tưởng chính

- Lượt thứ nhất, chọn trong dãy khoá $k[1..n]$ ra khoá nhỏ nhất và đổi giá trị với $k[1]$, khi đó $k[1]$ sẽ trở thành khoá nhỏ nhất.
- Lượt thứ hai, chọn trong dãy khoá $k[2..n]$ ra khoá nhỏ nhất và đổi giá trị với $k[2]$.
- ...
- Lượt $n-1$, chọn giá trị nhỏ nhất trong $k[n-1]$ và $k[n]$ ra khoá nhỏ nhất và đổi giá trị với $k[n-1]$.

2. Các bước thực hiện

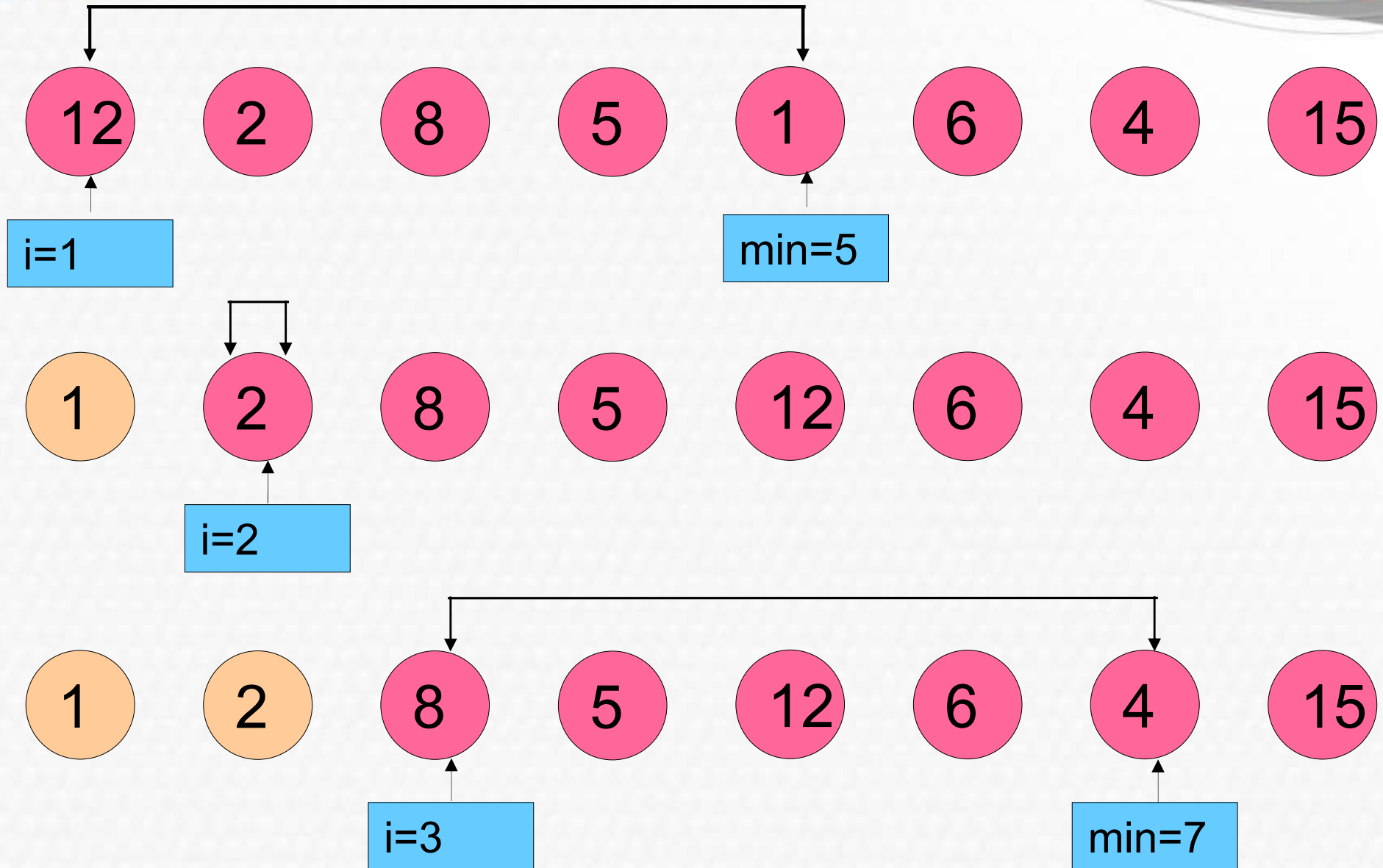
- **B1**: $i = 1$
- **B2**: Tìm phần tử $a[\text{min}]$ nhỏ nhất trong dãy hiện hành từ $a[i]$ đến $a[n]$
- **B3**: Hoán vị $a[i]$ và $a[\text{min}]$
- **B4**: Nếu $i < n - 1$ thì $i = i + 1 \Rightarrow$ Lặp B2
Ngược lại $\Rightarrow$ Dừng

Ví dụ 6.1: Cho dãy số như sau:

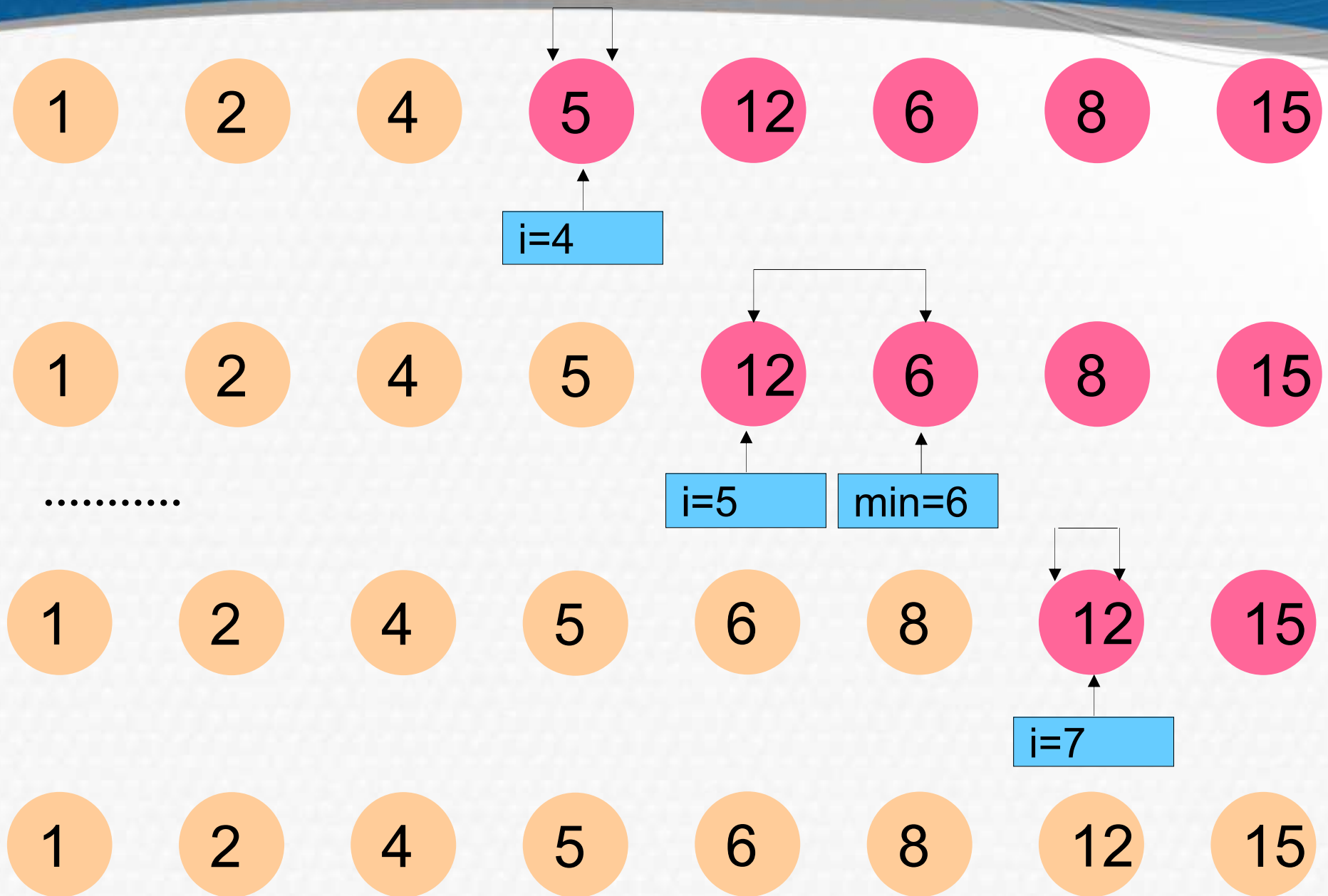
12 2 8 5 1 6 4 15

Minh họa phương pháp chọn như sau:

6.1. SẮP XẾP



6.1. SẮP XẾP



3. Cài đặt Selection Sort

```
void SelectionSort(int a[], int n)
{
    int min;                // lưu chỉ số phần tử nhỏ nhất
    for(int i = 0; i < n-1; i++)    // duyệt qua n-1 phần tử
    {
        min = i;
        for (int j = i+1; j < n; j++)
            if (a[j] < a[min]) min = j;
        swap(a[min], a[i]);
    }
}
```

6.1. SẮP XẾP

6.1.2. Bubble Sort (sắp xếp nổi bọt)

1. Ý tưởng chính: Xem các số cần sắp là trọng lượng của các bọt khí trong nước, bọt khí nào nhẹ sẽ nổi lên trước. Ý tưởng thuật toán như sau:

- Xuất phát từ cuối dãy, **đổi chỗ các cặp phần tử kế cận** để đưa phần tử nhỏ hơn về đầu.
- Sau đó ở bước tiếp theo không xét phần tử đó nữa. Do vậy lần **xử lý thứ i sẽ có vị trí đầu dãy là i .**
- Lặp lại xử lý trên cho đến khi không còn cặp phần tử nào được xét.



2. Các bước tiến hành

- B1: $i = 1$; // lần xử lý đầu tiên
- B2: $j = n$; // duyệt từ cuối dãy ngược về vị trí i

Trong khi ($j > i$) thực hiện:

Nếu $a[j] < a[j-1]$: Hoán đổi $a[j]$ và $a[j-1]$

$j = j - 1$;

- B3: $i = i + 1$; // lần xử lý kế tiếp

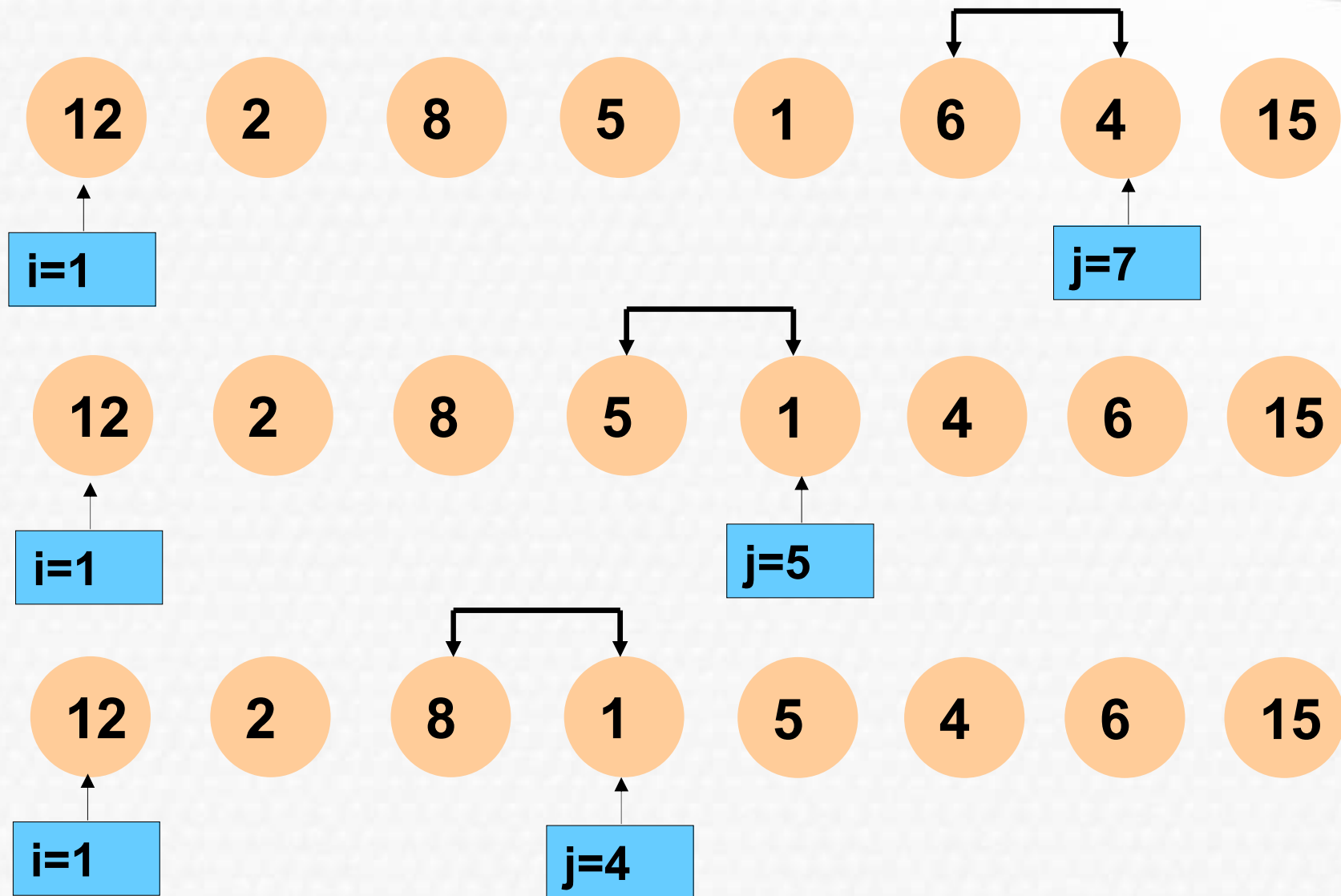
Nếu $i > n - 1$: Hết dãy $\Rightarrow$ Dừng

Ngược lại: quay lại B2

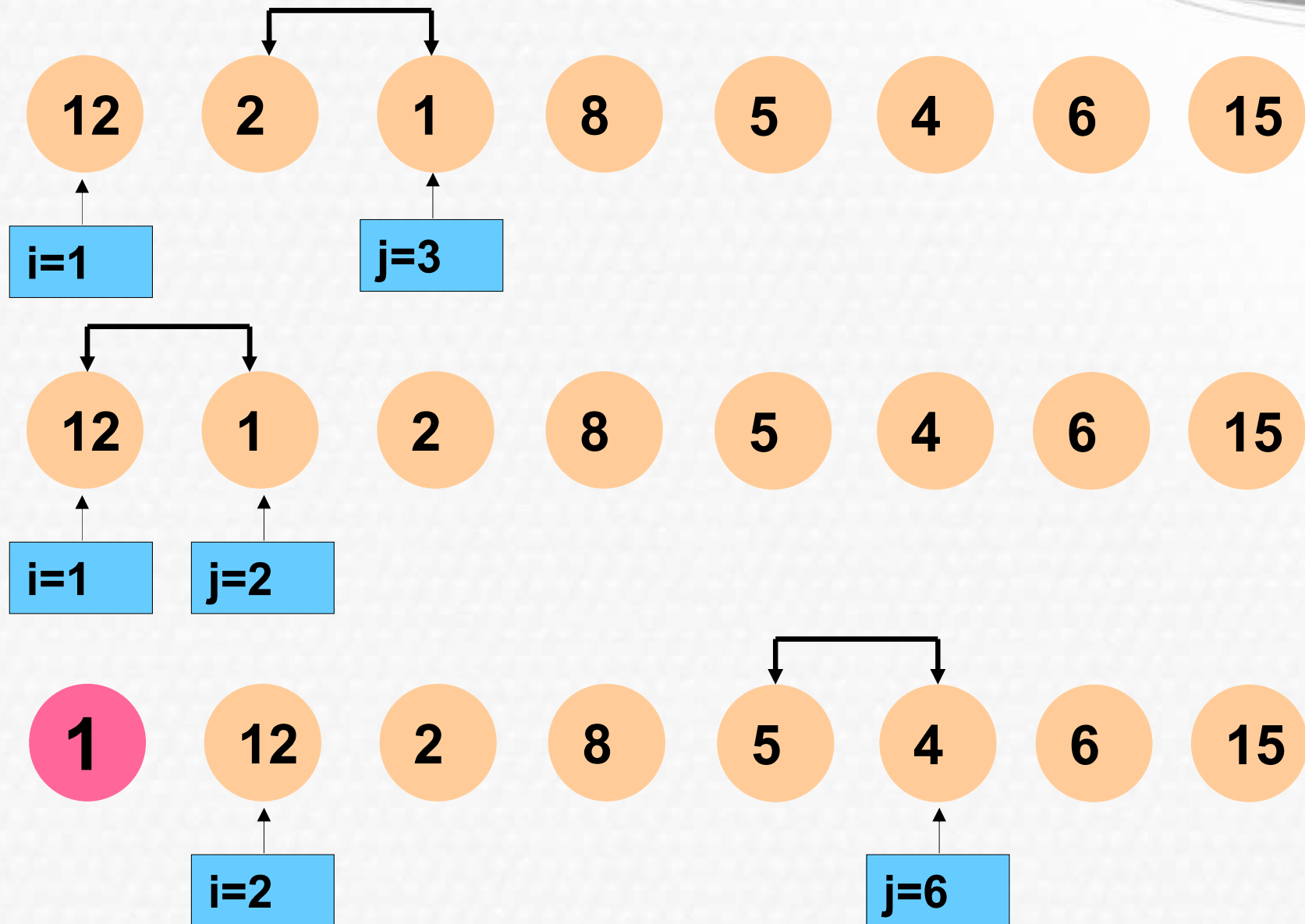
Ví dụ 6.2: Minh họa sắp xếp dãy số sau:

12 2 8 5 1 6 4 15

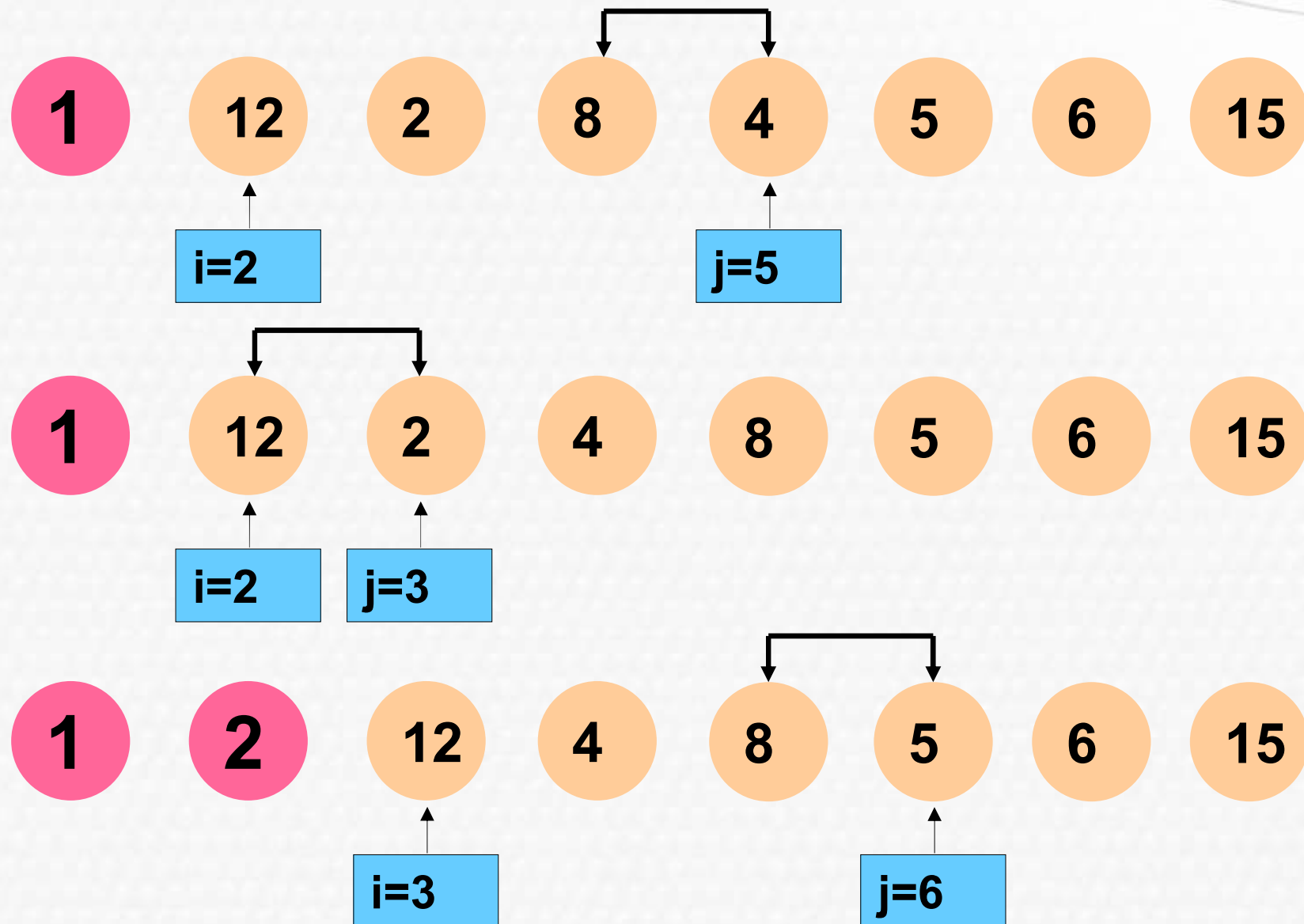
6.1. SẮP XẾP



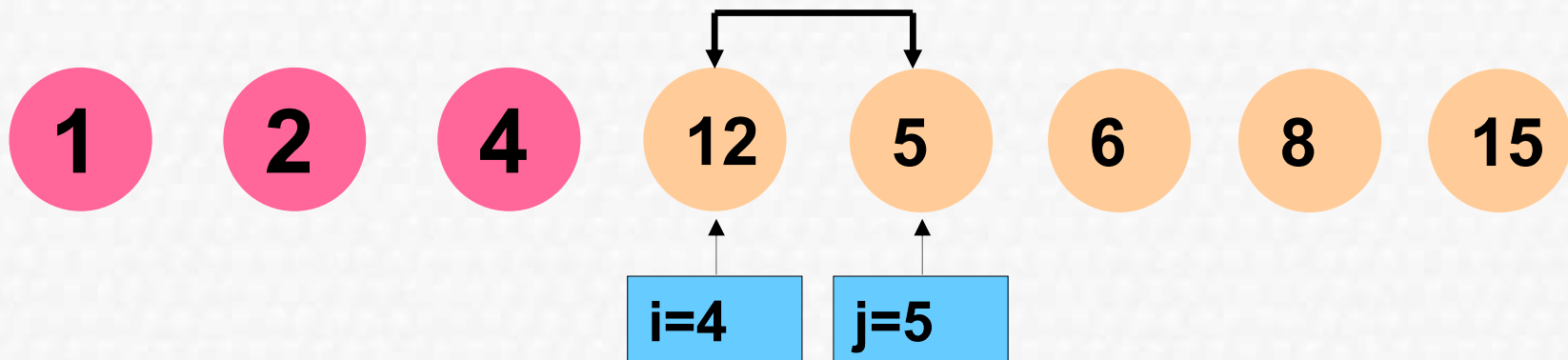
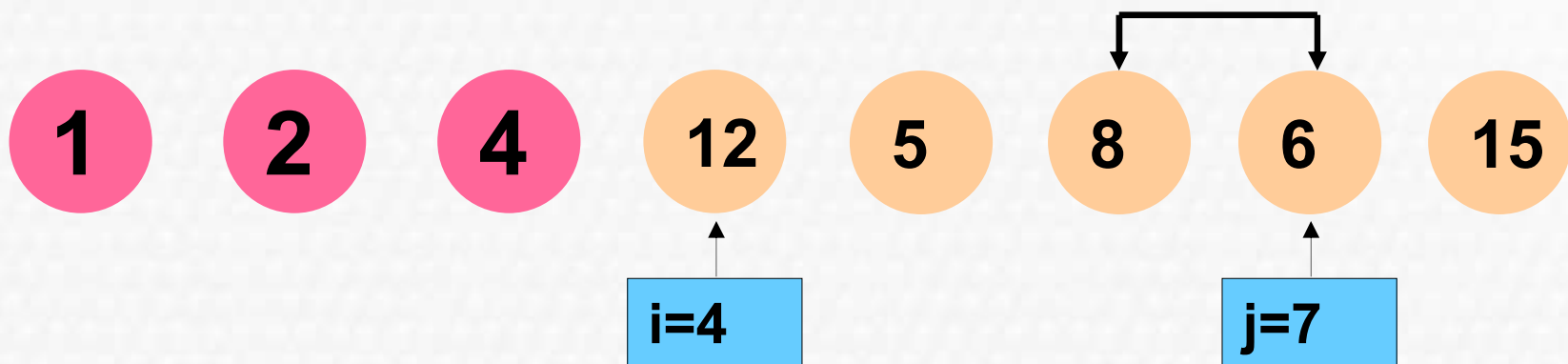
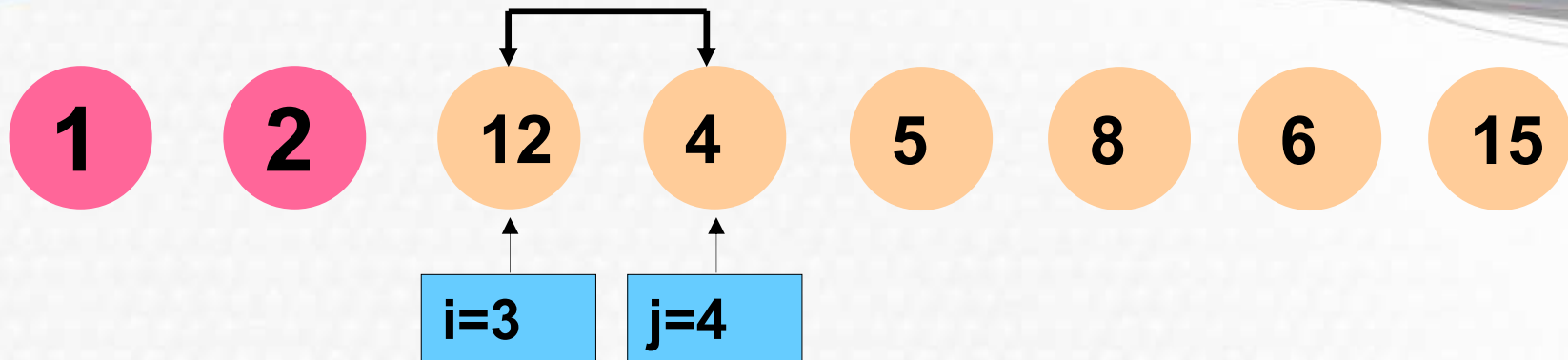
6.1. SẮP XẾP



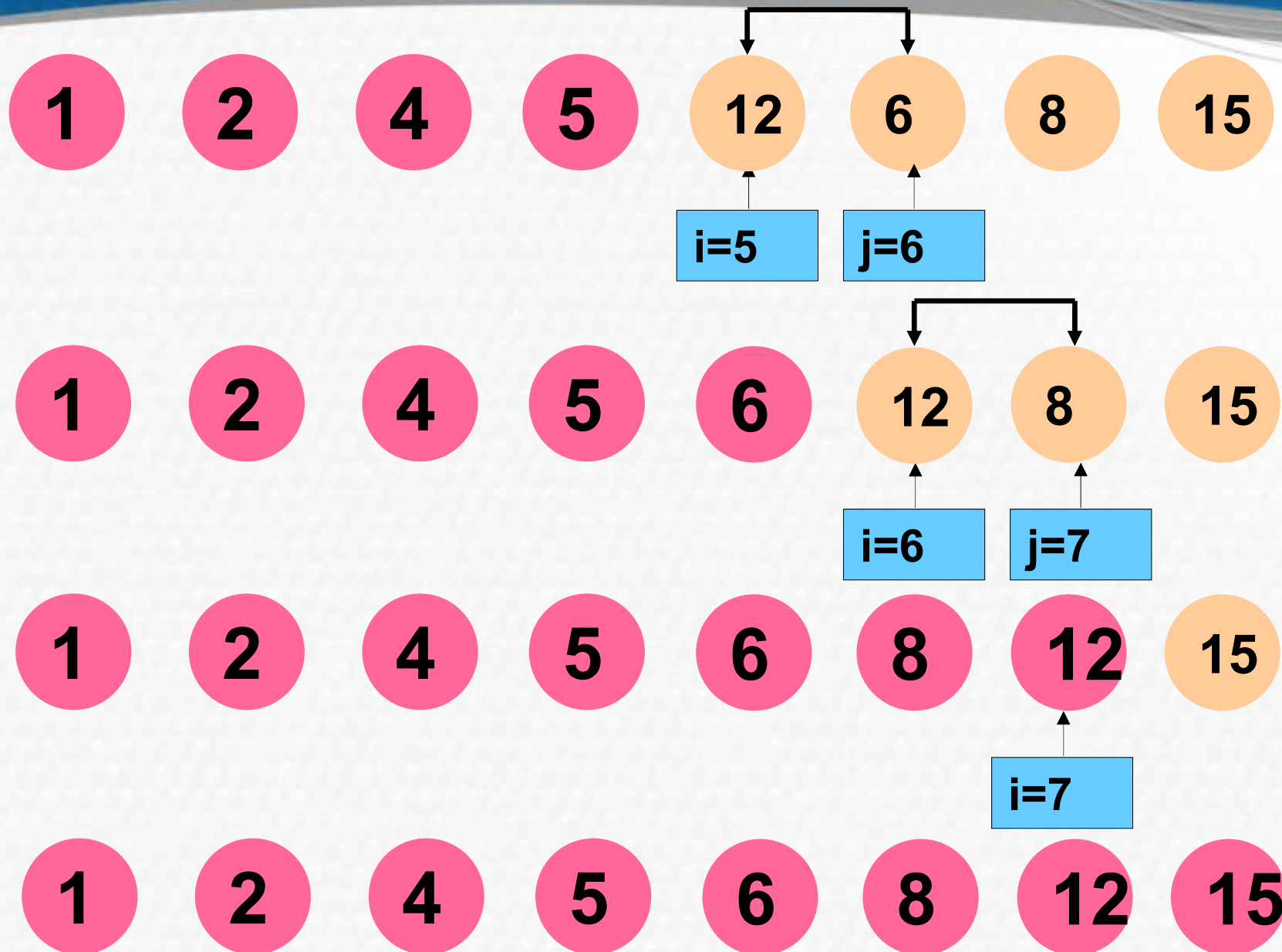
6.1. SẮP XẾP



6.1. SẮP XẾP



6.1. SẮP XẾP



3. Cài đặt Bubble sort

```
void BubbleSort(int a[], int n)
{
    int i, j;
    for (i = 1; i < n-1; i++)
        for(j = n-1; j >= i; j--)
            if (a[j] < a[j-1])
                Swap(a[j], a[j-1]);
}
```

6.1. SẮP XẾP

Ta nhận thấy phương pháp Bubble có thể được cải tiến như sau:

Nếu ở lần duyệt dãy nào đó mà không có có sự đổi chỗ giữa hai phần tử thì:

- 🌸 Dãy đã có thứ tự $\Rightarrow$ giải thuật kết thúc.
- 🌸 Ta dùng một cờ hiệu **flag** để ghi nhận điều này
- 🌸 Giải thuật Bubble Sort được cải tiến như sau:

6.1. SẮP XẾP

```
#define FALSE 0
#define TRUE 1
void Bubble_Sort_Ad(int A[], int n)
{  int i, j;
   int flag = TRUE;
   for (i = 0; i < n-1 && flag == TRUE; i++)
       {  flag = FALSE ;
          for (j = n-1; j > n-1; j--)
              if (a[j] < a[j-1])
                  {  flag = TRUE;
                     Swap(a[j], a[j-1]);
                  }
       }
}
```

6.1. SẮP XẾP

6.1.3. Insertion Sort (sắp xếp chèn)

1. Ý tưởng chính: Phỏng theo cách làm của người chơi bài khi cần “**chèn**” thêm 1 con bài vào bộ bài đã được sắp xếp trên tay.

- Cho dãy ban đầu $a[1], a[2], \dots, a[n]$, ta có thể xem dãy con gồm một phần tử $a[1]$ đã được sắp.
 - Sau đó thêm $a[2]$ vào đoạn $a[1]$ sao cho $a[1] a[2]$ được sắp.
 - Tiếp tục thêm $a[3]$ vào để có $a[1] a[2] a[3]$ được sắp....
 - Cho đến khi thêm xong $a[n]$ vào đoạn $a[1] a[2] \dots a[n-1]$.
- $\Rightarrow$ đoạn $a[1] a[2] \dots a[n-1] a[n]$ được sắp.



2. Các bước tiến hành

– **B1**: $i = 2$; //giả sử có đoạn $a[1]$ đã được sắp

– **B2**: $x = a[i]$;

Tìm được vị trí cần chèn x vào là **pos**

– **B3**: Dời chỗ các phần tử từ $a[pos] \Rightarrow a[i-1]$ sang phải một vị trí để dành chỗ cho $a[i]$.

– **B4**: $a[pos] = x$; // có đoạn $a[1]...a[i]$ được sắp.

– **B5**: $i = i + 1$;

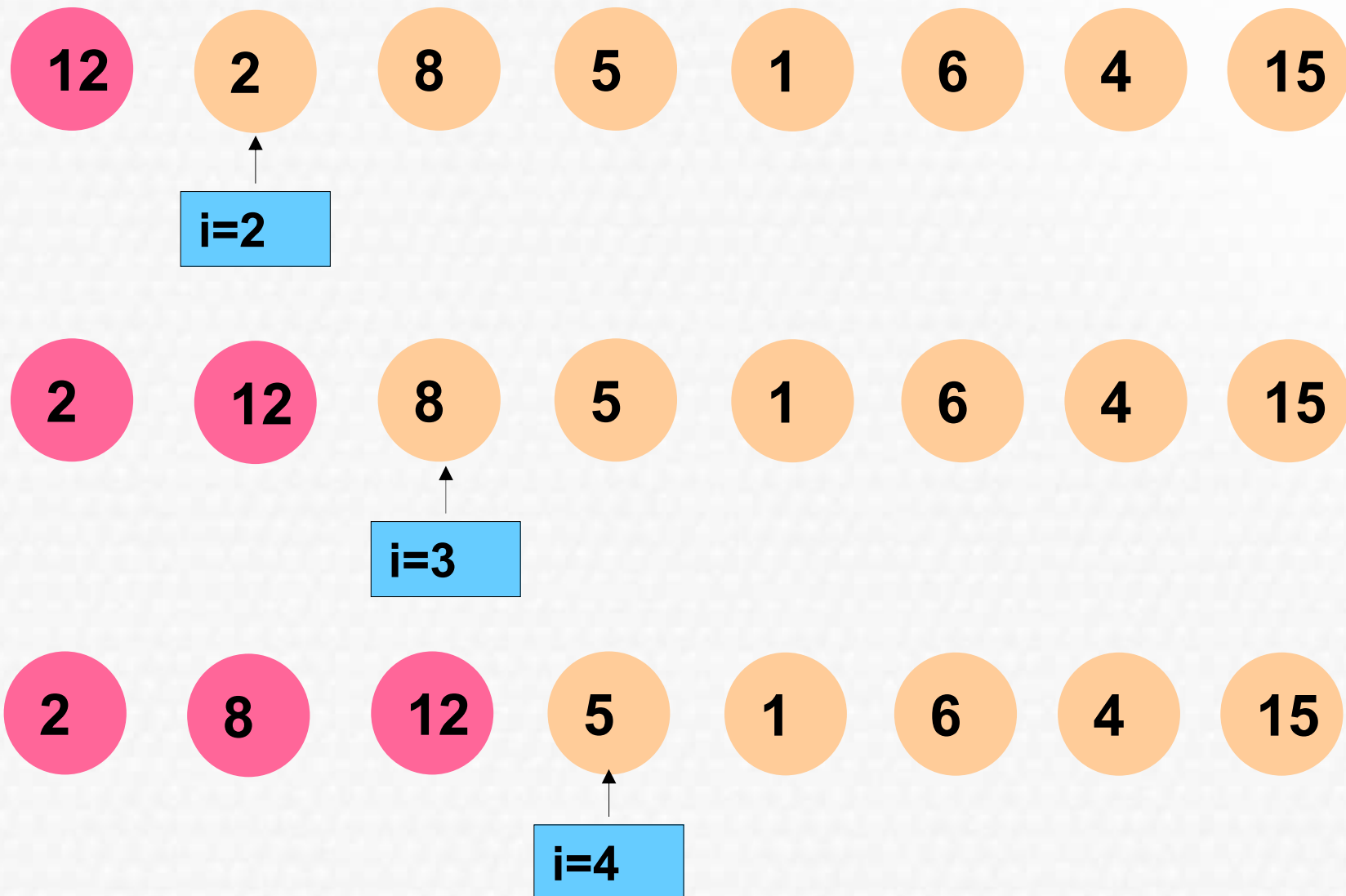
Nếu $i \leq n$: Lặp lại B2

Ngược lại: Dừng $\Rightarrow$ Dãy đã được sắp

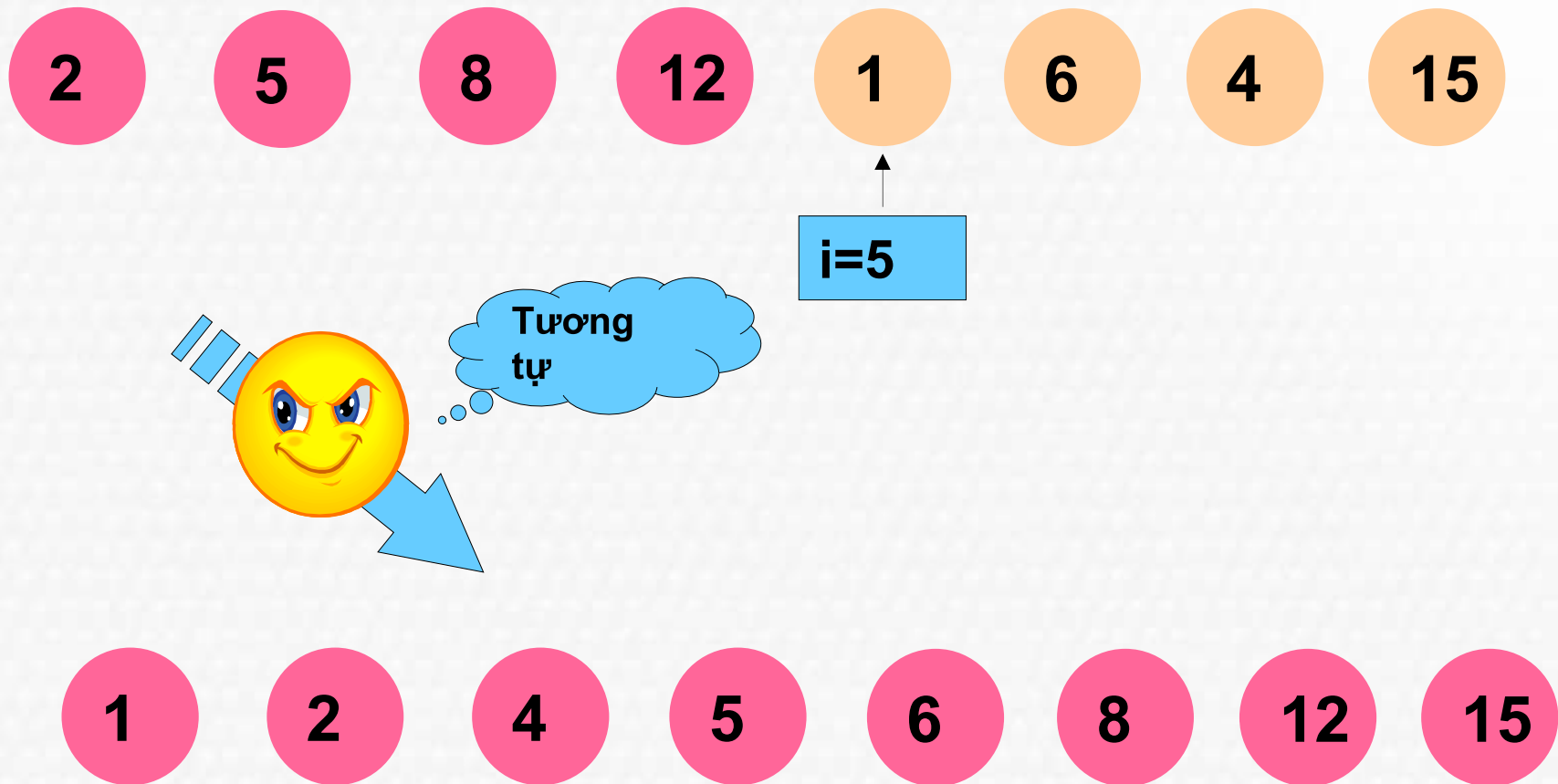
Ví dụ 6.3: Minh họa phương pháp chèn với dãy:

12 2 8 5 1 6 4 15

6.1. SẮP XẾP



6.1. SẮP XẾP



3. Cài đặt Insertion sort

```
void InsertionSort(int a[], int n)
```

```
{  int pos, i, x;           // x lưu phần tử a[i]
    for (i = 1; i < n; i++)
    {  x = a[i]; pos = i-1;
        while ((pos >= 0) && (a[pos] > x))
        { // kết hợp dời chỗ các phần tử đứng sau x trong dãy mới
            a[pos+1] = a[pos];
            pos--;
        }
        a[pos+1] = x; // chèn x vào dãy mới
    }
}
```

6.1.4. QuickSort (sắp xếp nhanh)

- Selection Sort, Insertion Sort, Bubble Sort có độ phức tạp trong trường hợp xấu nhất là $O(n^2)$.
- Độ phức tạp do số lần so sánh và hoán đổi lớn, thực hiện trên toàn tập các phần tử cần sắp
- ➔ Khắc phục bằng kỹ thuật **Chia để trị** (Divide and Conquer): Chia dãy cần sắp thành các dãy con, tiến hành sắp xếp trên các dãy con.
- ➔ Giảm số phần tử cần so sánh và hoán đổi.

QuickSort do Hoare đề xuất năm 1960, viết tắt là QS.

- ▶ Tốc độ trung bình nhanh hơn thuật toán khác
- ▶ Do đó Hoare dùng “*quick*” để đặt tên

1. Ý tưởng chính

- QS là thuật toán được phát triển dựa trên kỹ thuật chia để trị.
- QS phân hoạch dãy ban đầu thành hai phần dựa vào một giá trị x .
 - Dãy 1: gồm các phần tử $a[i]$ “ $\leq$ ” x
 - Dãy 2: gồm các phần tử $a[i]$ “ $\geq$ ” x
- Sau khi phân hoạch thì dãy ban đầu được phân thành ba phần:
 - $a[k] < x$, với $k = 1..i$
 - $a[k] = x$, với $k = i..j$
 - $a[k] > x$, với $k = j..n$

$a[k] < x$

$a[k] = x$

$a[k] > x$

6.1. SẮP XẾP

2. Giải thuật phân hoạch dãy $a[\text{left}], a[\text{left}+1], \dots, a[\text{right}]$ thành hai dãy con:

B1: Chọn tùy ý phần tử $a[k]$ trong dãy là **giá trị mốc**, $\text{left} \leq k \leq \text{right}$,
– Cho $x = a[k]$, $i = \text{left}$, $j = \text{right}$.

B2: Tìm và hoán vị cặp phần tử $a[i]$ và $a[j]$ không đúng thứ tự đang sắp.

- **B2-1:** Trong khi $a[i] < x \Rightarrow i++$;
- **B2-2:** Trong khi $a[j] > x \Rightarrow j--$;
- **B2-3:** Nếu $i < j \Rightarrow \text{swap}(a[i], a[j])$ // $a[i], a[j]$ sai thứ tự

B3:

- Nếu $i < j \Rightarrow$ Bước 2;
- Nếu $i \geq j \Rightarrow$ Dừng.

6.1. SẮP XẾP

- Giải thuật để sắp xếp dãy $a[\text{left}], a[\text{left}+1], \dots, a[\text{right}]$: được phát biểu theo cách đệ quy như sau:

B1: Phân hoạch dãy $a[\text{left}] \dots a[\text{right}]$ thành các dãy con:

- Dãy con 1: $a[\text{left}] \dots a[j] < x$
- Dãy con 2: $a[j+1] \dots a[i-1] = x$
- Dãy con 3: $a[i] \dots a[\text{right}] > x$

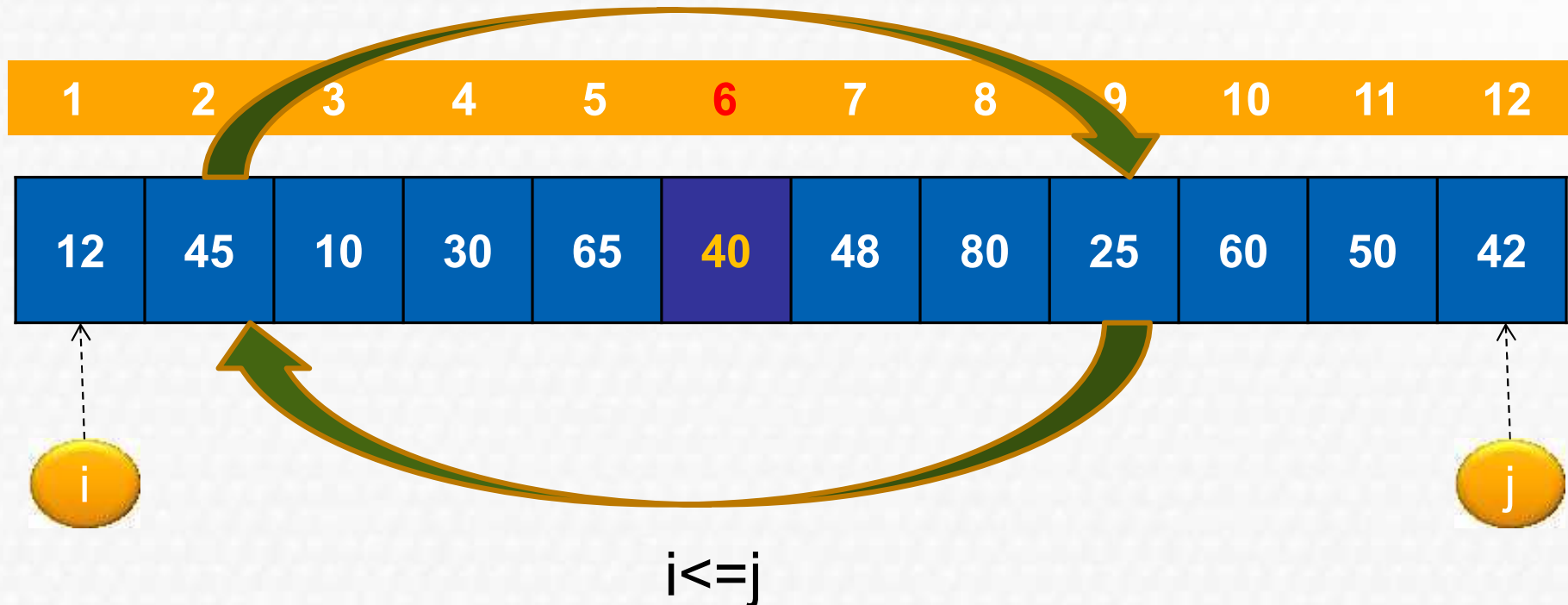
B2:

- Nếu $(\text{left} < j)$ // dãy con 1 có nhiều hơn 1 phần tử
Phân hoạch dãy $a[\text{left}] \dots a[j]$
- Nếu $(i < \text{right})$ // dãy con 3 có nhiều hơn 1 phần tử
Phân hoạch dãy $a[i] \dots a[\text{right}]$

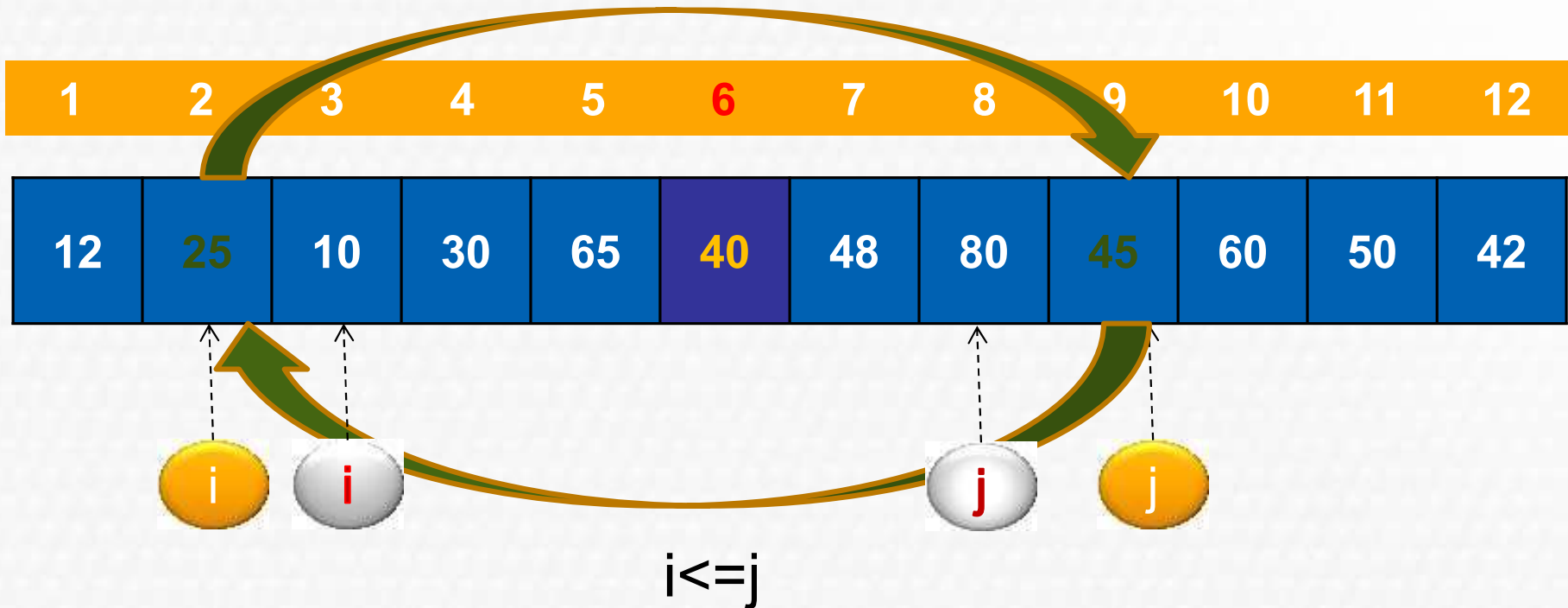
6.1. SẮP XẾP

Ví dụ 6.4: Minh họa sắp xếp dãy số sau:

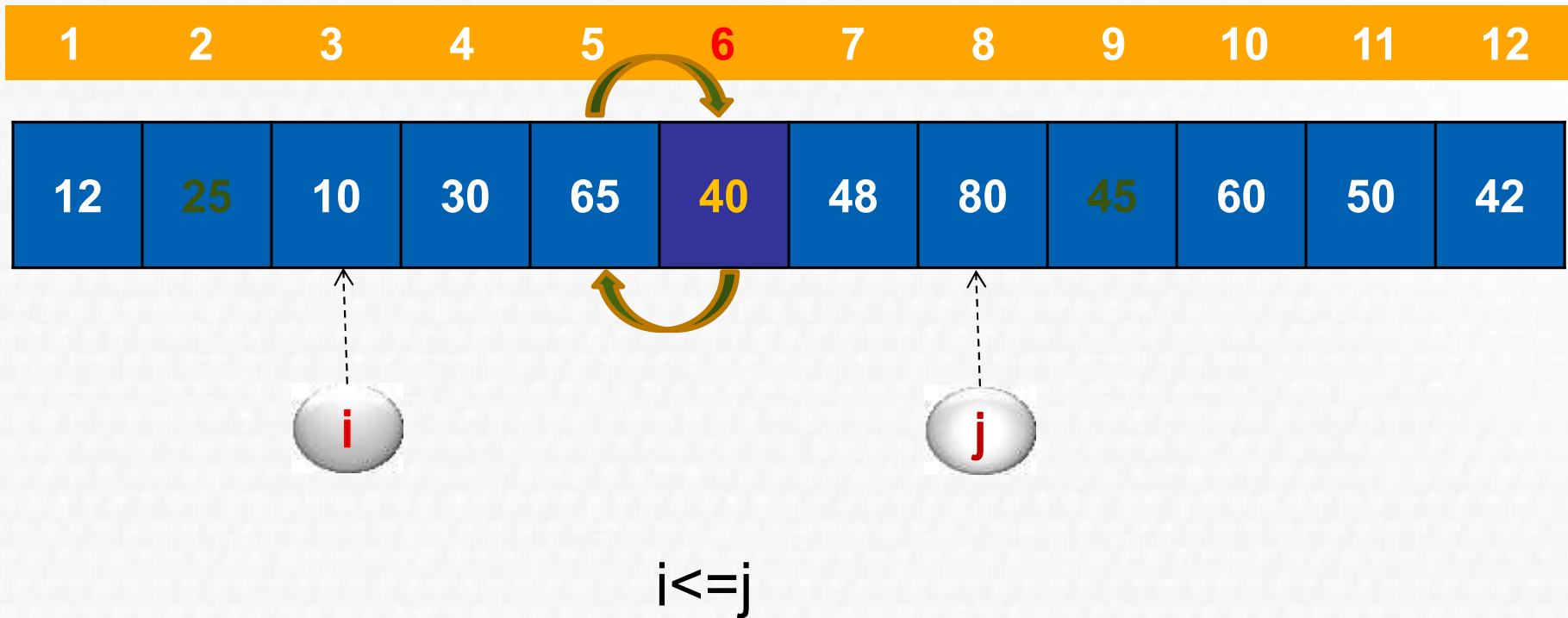
12 45 10 30 65 40 48 80 25 60 50 42



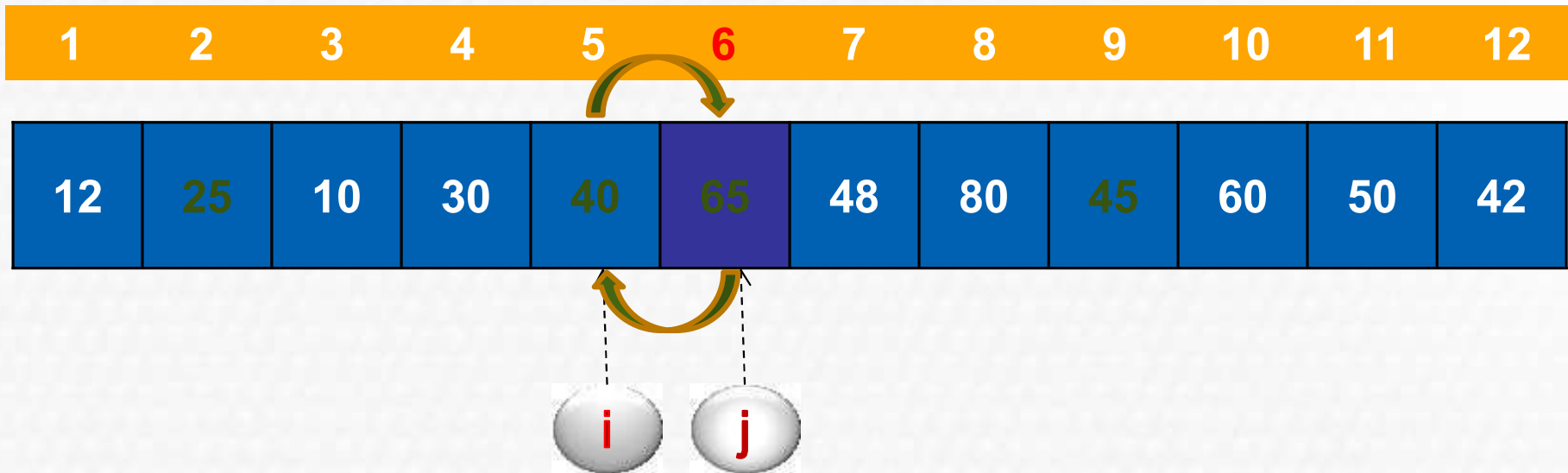
6.1. SẮP XẾP



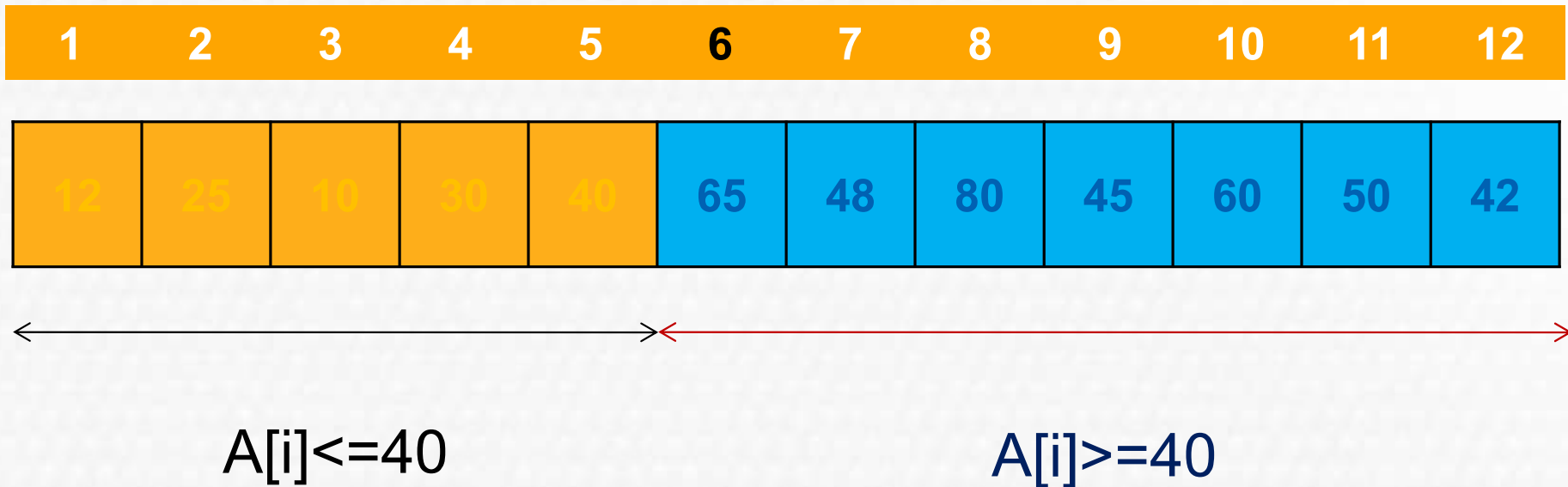
6.1. SẮP XẾP



6.1. SẮP XẾP

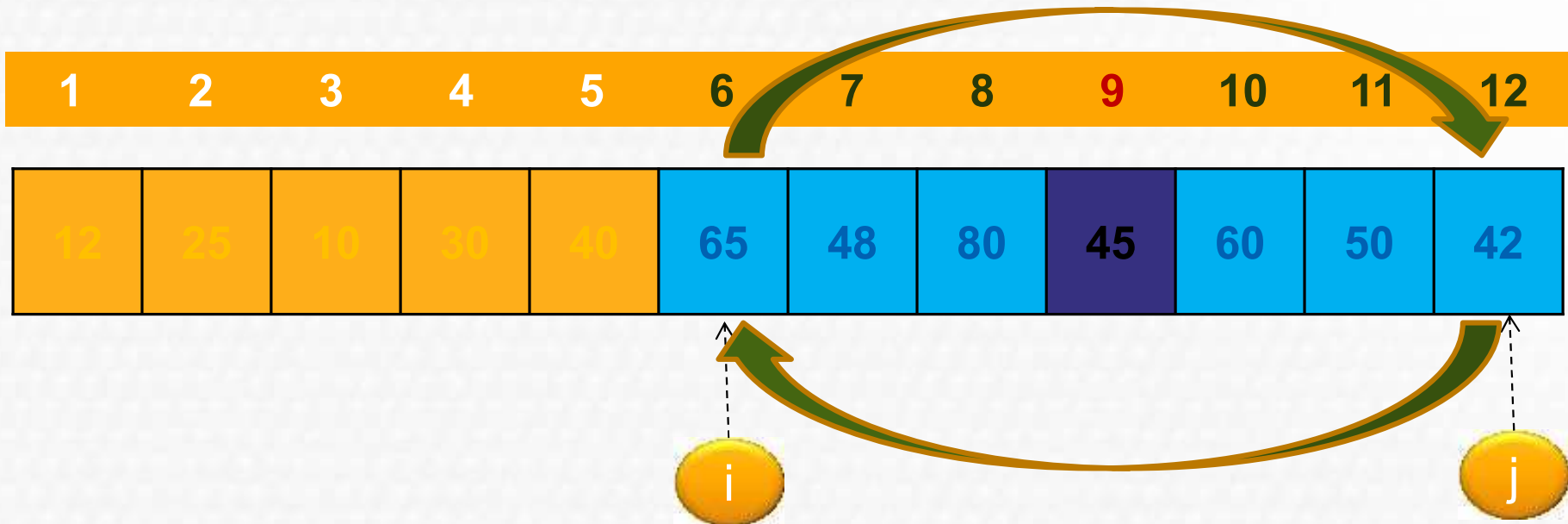


6.1. SẮP XẾP

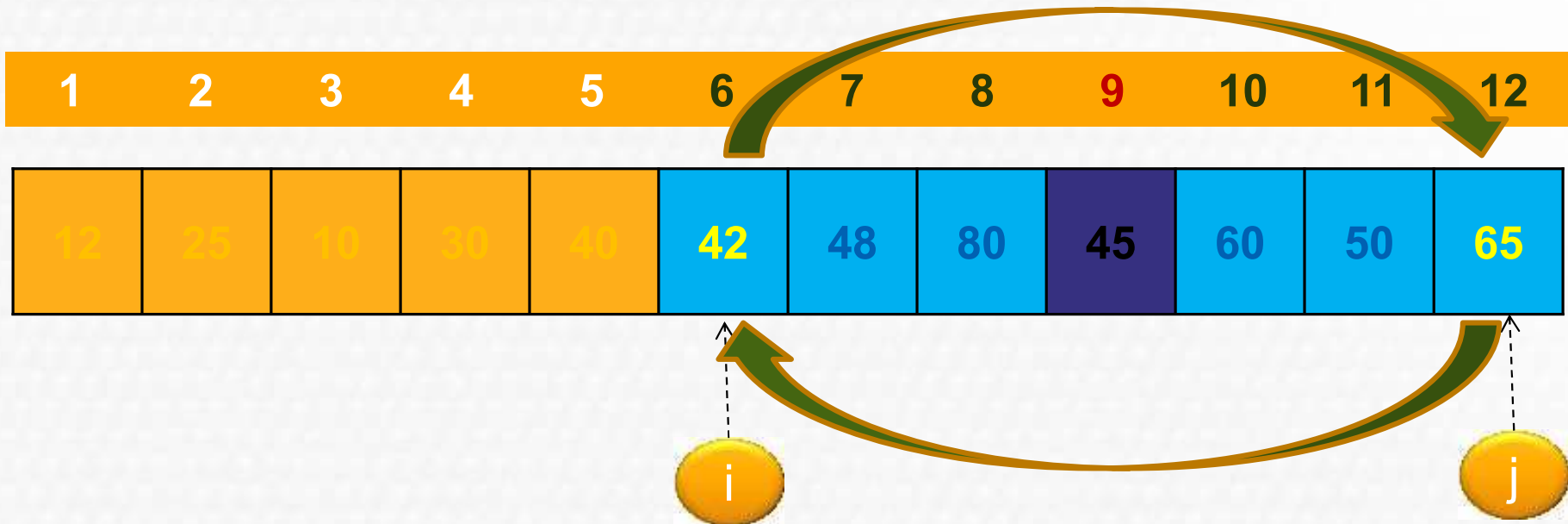


Tiến hành lại quá trình sắp xếp trên cho các đoạn con

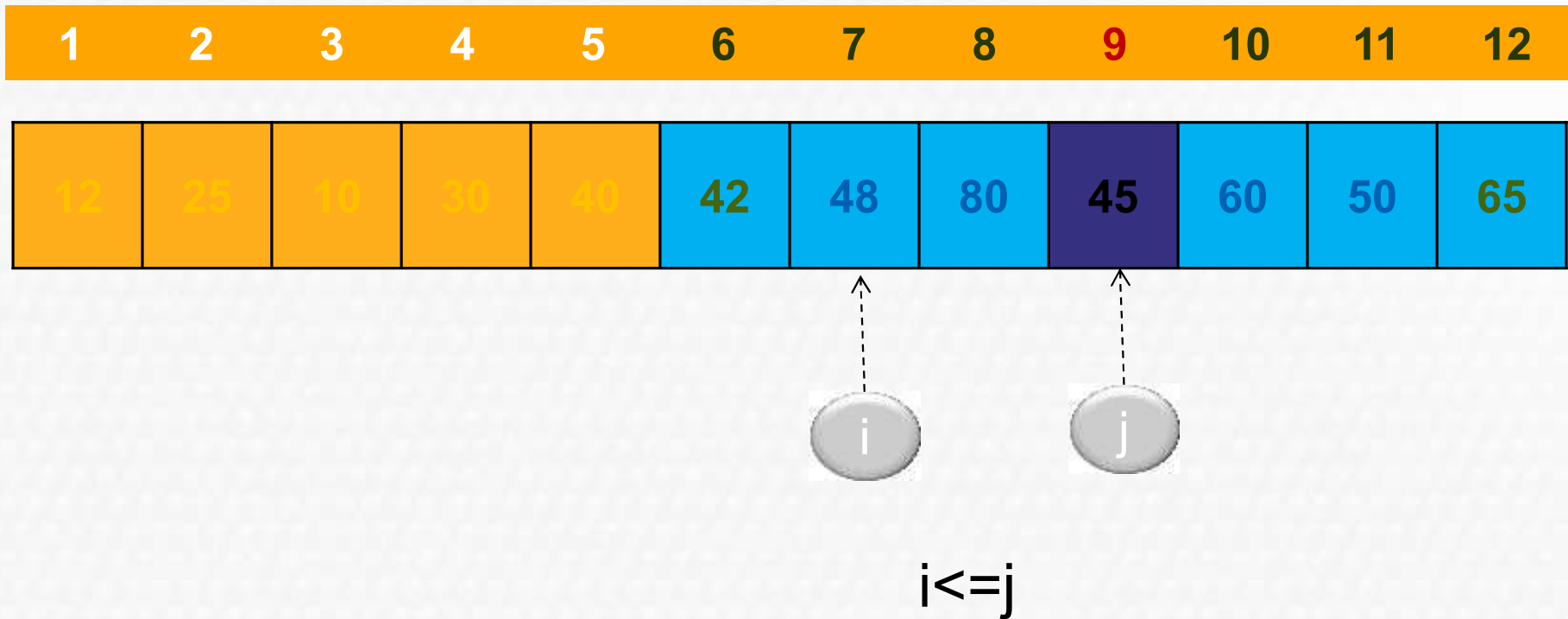
6.1. SẮP XẾP



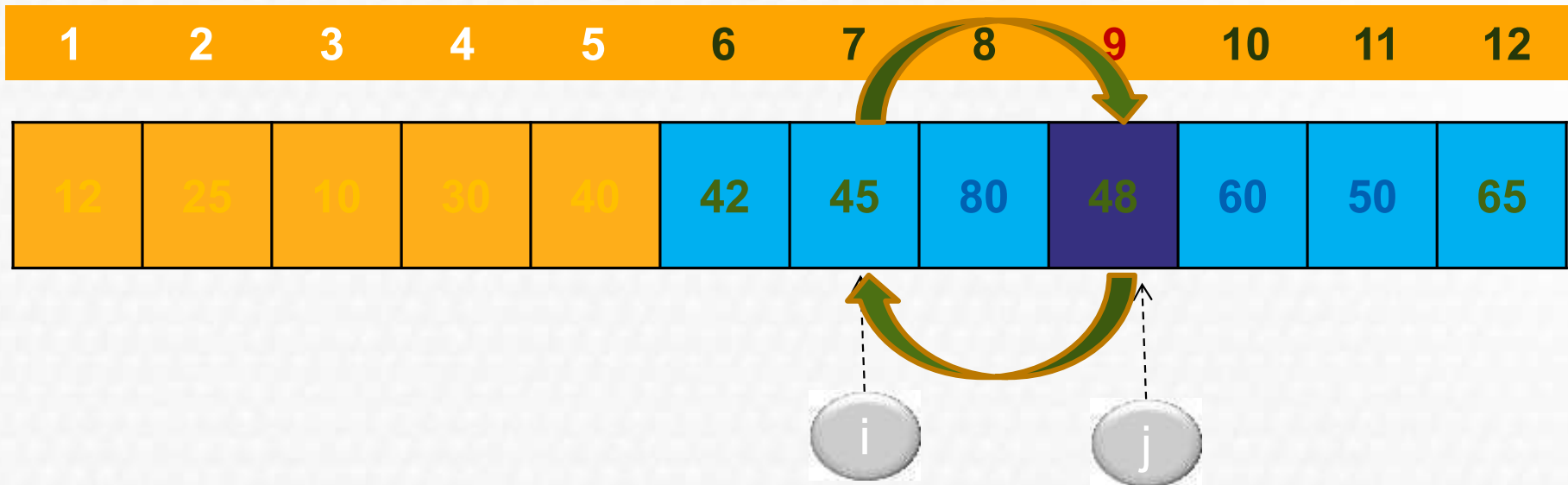
6.1. SẮP XẾP



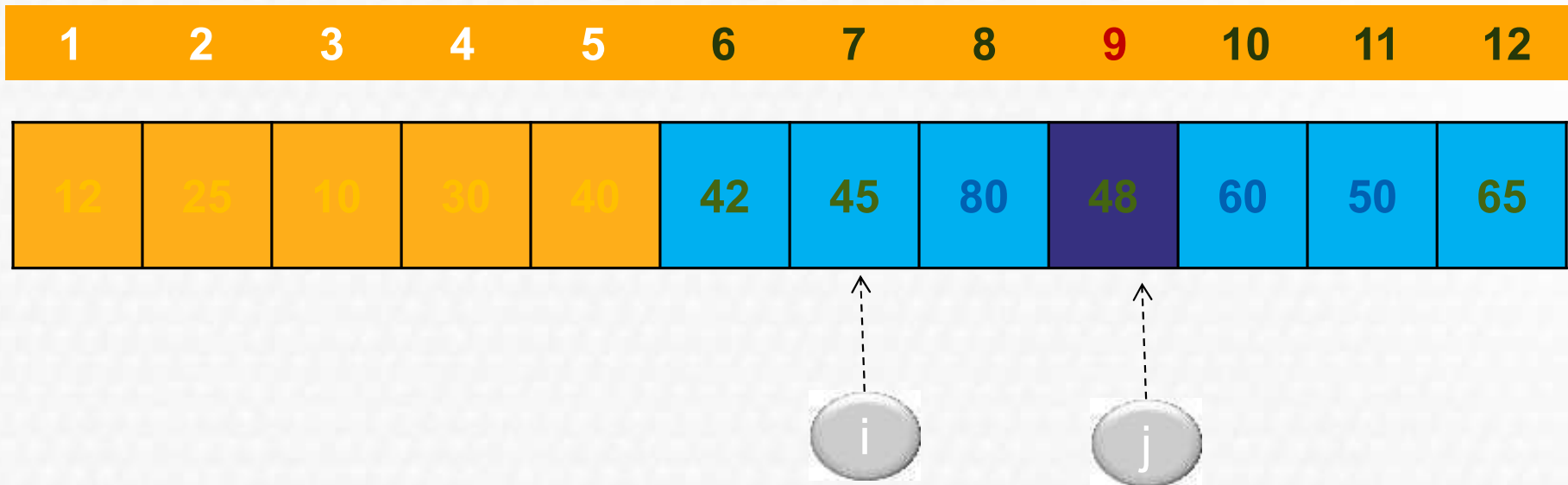
6.1. SẮP XẾP



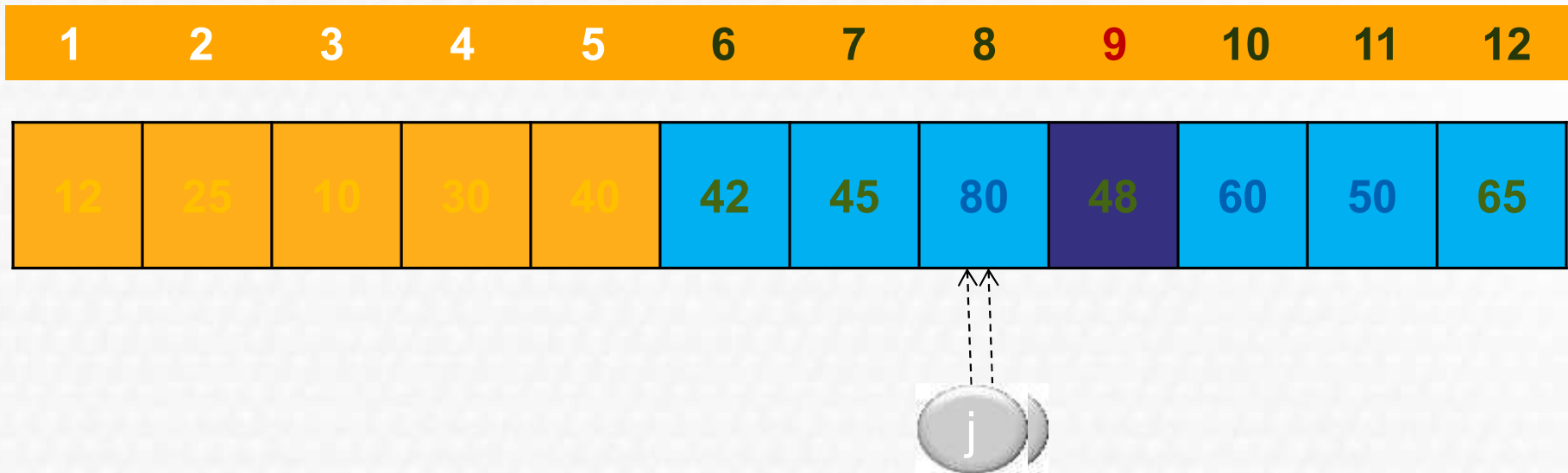
6.1. SẮP XẾP



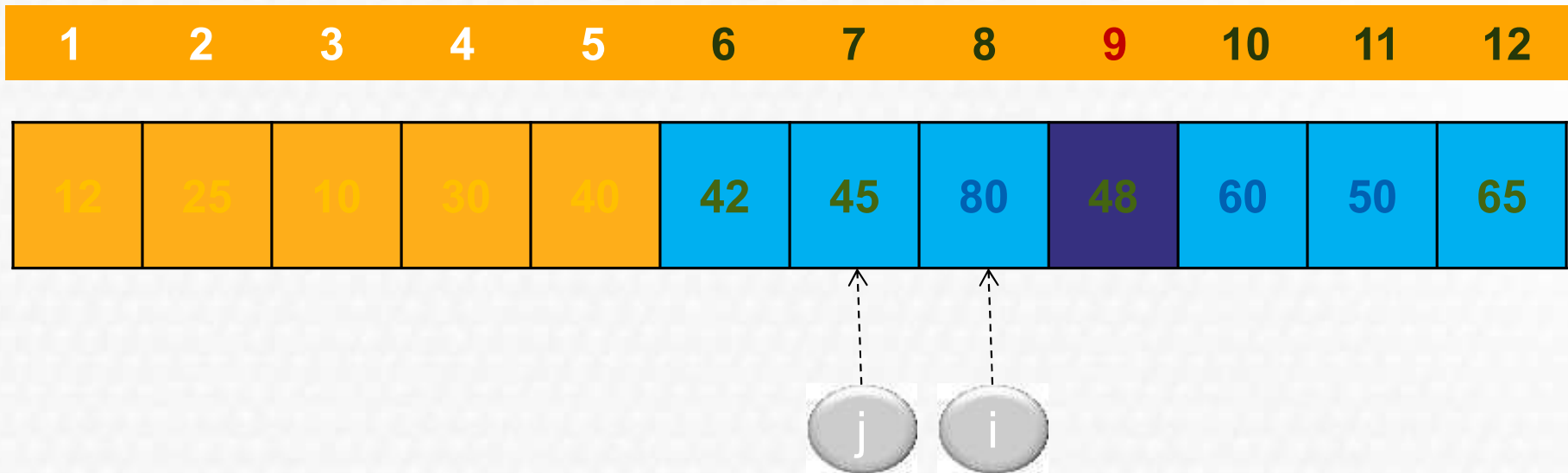
6.1. SẮP XẾP



6.1. SẮP XẾP

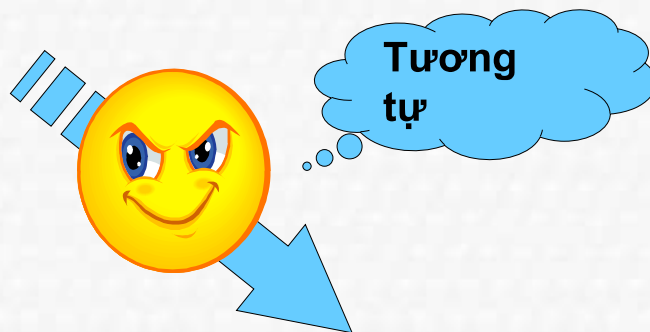


6.1. SẮP XẾP



6.1. SẮP XẾP

1	2	3	4	5	6	7	8	9	10	11	12
12	25	10	30	40	42	45	80	48	60	50	65



10	12	25	30	40	42	45	48	50	60	65	80
----	----	----	----	----	----	----	----	----	----	----	----

3. Cài đặt QuickSort

```
void QuickSort(int a[], int left, int right) {  
    int i, j, x;  
    x = a[(left+right)/2];           // chọn phần tử giữa làm gốc  
    i = left; j = right;  
    do {  
        while (a[i] < x) i++;         // lặp đến khi a[i] >= x  
        while (a[j] > x) j--;         // lặp đến khi a[j] <= x  
        if ( i <= j ) {  
            swap(a[i], a[j]);  
            i++;                      // qua phần tử kế tiếp  
            j--;                      // qua phần tử đứng trước  
        }  
    } while (i < j);  
    if (left < j) QuickSort(a, left, j); // ph đoạn bên trái  
    if (right > i) QuickSort(a, i, right); // ph đoạn bên phải  
}
```

6.1. SẮP XẾP

6.1.5 Tổng kết các thuật toán sắp xếp dựa trên phép so sánh

Tên phương pháp	Trung bình	Tối nhất
Bubble Sort	-	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n + d)$	$O(n^2)$
Quick Sort	$O(n \log n)$	$O(n^2)$

6.2. TÌM KIẾM

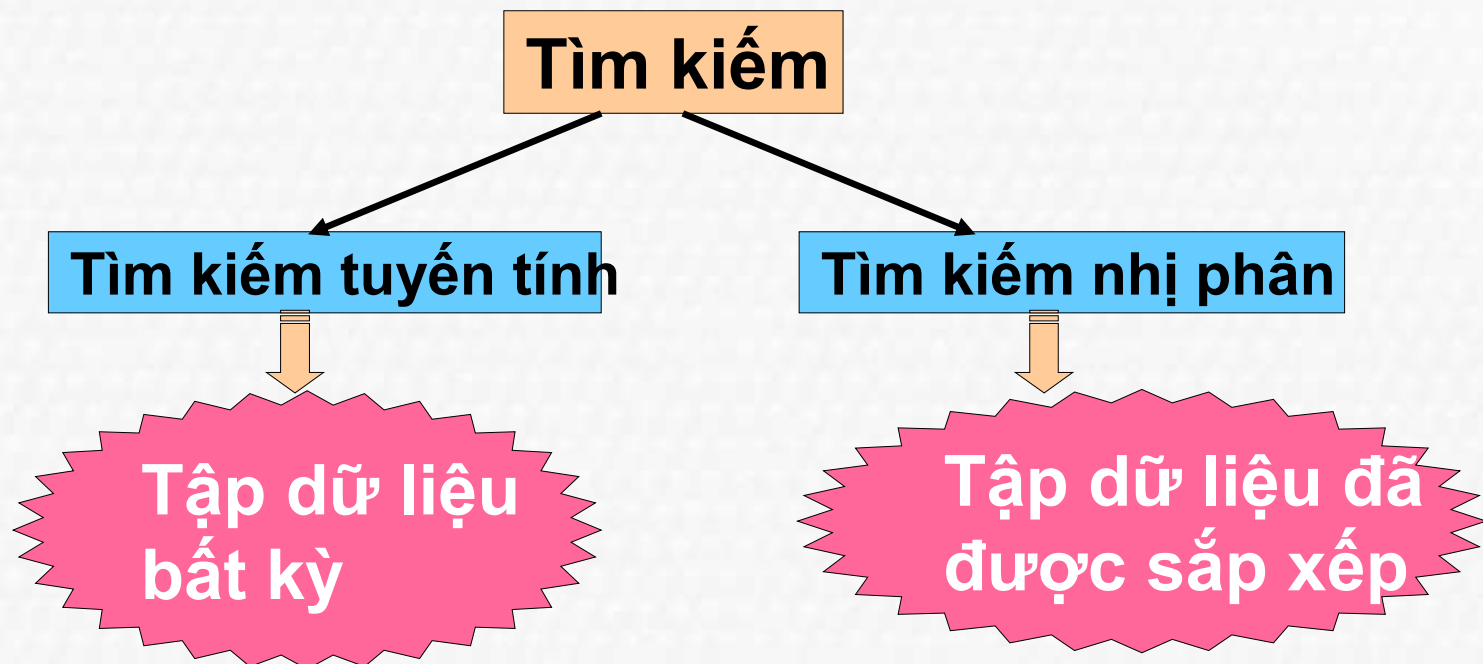
- Tìm kiếm là **thao tác quan trọng & thường xuyên** trong tin học.
 - Tìm kiếm một nhân viên trong danh sách nhân viên.
 - Tìm kiếm một tên sách trong thư viện.
- Tìm kiếm là **quá trình xác định một đối tượng nào đó trong một tập các đối tượng**. Kết quả trả về là đối tượng tìm được (*nếu có*) hoặc một chỉ số (*nếu có*) xác định vị trí của đối tượng trong tập đó.
- Việc tìm kiếm dựa theo một trường nào đó của đối tượng, trường này là **khóa** (key) của việc tìm kiếm.

VD: Đối tượng sinh viên có các dữ liệu {**MaSV**, **HoTen**, **DiaChi**,...}. Khi đó tìm kiếm trên danh sách sinh viên thì khóa thường chọn là **MaSV** hoặc **HoTen**.

6.2. TÌM KIẾM

■ Bài toán được mô tả như sau:

- Tập dữ liệu được lưu trữ là dãy $a_1, a_2, \dots, a_n$. Giả sử chọn cấu trúc dữ liệu mảng để lưu trữ dãy số này trong bộ nhớ chính, có khai báo: **int a[n];**
- Khóa cần tìm là **x**, có kiểu nguyên: **int x;**



6.2.1. Linear Search (tìm kiếm tuyến tính)

1. Ý tưởng chính: Duyệt tuần tự từ phần tử đầu tiên, lần lượt so sánh khóa tìm kiếm với khóa tương ứng của các phần tử trong danh sách. Cho đến khi gặp phần tử cần tìm hoặc đến khi duyệt hết danh sách.

2. Các bước tiến hành như sau:

- Bước 1: $i = 1$;
- Bước 2: So sánh $a[i]$ với x , có hai khả năng
 - $a[i] = x$: Tìm thấy. $\Rightarrow$ Dừng
 - $a[i] \neq x$: Sang bước 3
- Bước 3: $i = i + 1$ // xét phần tử kế tiếp trong mảng
 - Nếu $i > n$: Hết mảng, không tìm thấy. $\Rightarrow$ Dừng
 - Nếu $i \leq n$: Quay lại bước 2

6.2. TÌM KIẾM

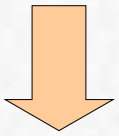
Ví dụ 6.5: Cho dãy số **a**, giá trị tìm **x = 8**:

12 2 5 8 1 6 4

Minh họa tìm kiếm tuyến tính

Tìm được

$x = 8$



3. Thuật toán tìm kiếm tuyến tính

```
int Search(int a[], int n, int key)
{
    int i = 0;
    while (i < n) && (key != a[i])
        i++;
    if (i >= n)
        return -1;           // tìm không thấy
    else
        return i;           // tìm thấy tại vị trí i
}
```

Thuật toán tìm kiếm tuyến tính cải tiến

```
int Search(int a[], int n, int key)
{
    int i = 0;
    a[n] = key;                // thêm phần tử thứ n+1
    while (key != a[i])
        i++;
    if (i == n)
        return -1;            // tìm hết mảng nhưng không có x
    else
        return i;             // tìm thấy x tại vị trí i
}
```

6.2. TÌM KIẾM



Nhận xét

- Giải thuật tìm kiếm tuyến tính **không phụ thuộc vào thứ tự** của các phần tử trong mảng, do vậy đây là phương pháp **tổng quát nhất** để tìm kiếm trên một dãy bất kỳ.
- Độ phức tạp: **$O(n)$** .
- Không đòi hỏi các số được sắp thứ tự.
- Làm việc được với cả danh sách liên kết.
- Một thuật toán có thể được cài đặt theo **nhiều cách khác nhau, kỹ thuật cài đặt ảnh hưởng nhiều đến tốc độ thực hiện**. Ví dụ như thuật toán Search cải tiến sẽ chạy nhanh hơn thuật toán trước do vòng lặp while chỉ so sánh một điều kiện...

6.2. TÌM KIẾM

6.2.2. Binary Search (Tìm kiếm nhị phân)

- Phép tìm kiếm nhị phân được áp dụng trên **dãy khóa đã có thứ tự**:

$$k[1] \leq k[2] \leq \dots \leq k[n].$$

- Điều kiện để thực hiện thuật toán tìm kiếm nhị phân là:
 - ✓ Danh sách phải được sắp **thứ tự không giảm**;
 - ✓ Phải cho phép **trực truy**.

⇒ Tìm kiếm nhị phân không dùng được cho danh sách liên kết.



1. Ý tưởng

- Giả sử ta cần tìm trong đoạn $a[\text{left}, \dots, \text{right}]$ với khóa tìm kiếm là x , trước hết ta xét phần tử giữa là $a[\text{mid}]$, với $\text{mid} = [\text{left} + \text{right}] / 2$.
 - Nếu $a[\text{mid}] < x \rightarrow$ ta tiến hành tìm kiếm từ $a[\text{mid} + 1]$ đến $a[\text{right}]$.
 - Nếu $a[\text{mid}] > x \rightarrow$ ta tiến hành tìm kiếm từ $a[\text{left}]$ đến $a[\text{mid} - 1]$.
 - Nếu $a[\text{mid}] = x$ thì việc tìm kiếm thành công.
- Quá trình tìm kiếm **thất bại nếu $\text{left} > \text{right}$** .

2. Các bước tiến hành

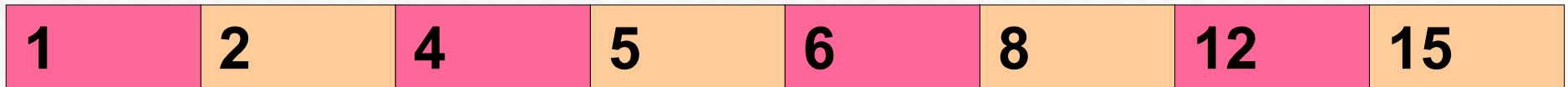
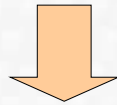
- **B1**: $\text{left} = 1, \text{right} = n$ // tìm kiếm trên tất cả phần tử
- **B2**: $\text{mid} = (\text{left} + \text{right})/2$ // lấy mốc so sánh
So sánh $a[\text{mid}]$ với x , có 3 khả năng
 - $a[\text{mid}] = x$: Tìm thấy $\Rightarrow$ Dừng
 - $a[\text{mid}] > x$: // tìm tiếp trong dãy $a[\text{left}] \dots a[\text{mid}-1]$
 $\text{right} = \text{mid} - 1;$
 - $a[\text{mid}] < x$: // tìm tiếp trong dãy $a[\text{mid}+1] \dots a[\text{right}]$
 $\text{left} = \text{mid} + 1$
- **B3**:
 - Nếu $\text{left} \leq \text{right}$ // còn phần tử $\Rightarrow$ tìm tiếp $\Rightarrow$ Lặp B2
 - Ngược lại: Dừng // đã xét hết các phần tử

6.2. TÌM KIẾM

Ví dụ 6.6: Cho dãy số gồm 8 phần tử bên dưới và $x = 8$:

1 2 4 5 6 8 12 15

X = 8



Left = 1

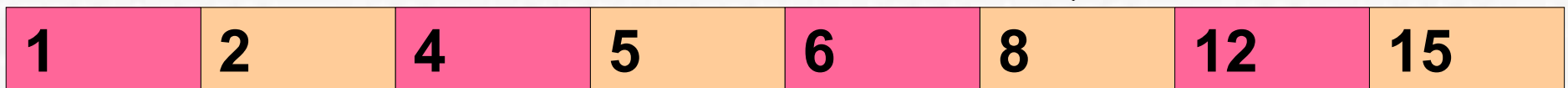
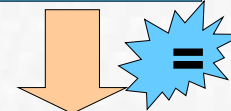
Mid = 4

Right = 8



Đoạn tìm kiếm

X = 8



Left = 5

Mid = 6

Right = 8



Đoạn tìm kiếm

3. Thuật toán Binary Search

```
int BinarySearch(int key) {  
    int left = 0, right = n-1, mid;  
    while (left <= right) {  
        mid = (left + right)/ 2; // lấy điểm giữa  
        if (a[mid] == key) // nếu tìm được  
            return mid;  
        if (a[mid] < key) // tìm đoạn bên phải mid  
            left = mid+1;  
        else // tìm đoạn bên trái mid  
            right = mid-1;  
    }  
    return -1; // không tìm được  
}
```



Nhận xét

- Thuật giải nhị phân dựa vào **quan hệ giá trị của các phần tử trong mảng** để định hướng trong quá trình tìm kiếm, do vậy chỉ áp dụng được với **dãy đã có thứ tự**.
- Độ phức tạp: **$O(\log n)$**
- Thuật giải nhị phân **tìm kiếm nhanh hơn tìm kiếm tuyến tính**.
- Tuy nhiên khi áp dụng thuật giải nhị phân thì cần phải quan tâm đến **chi phí cho việc sắp xếp mảng**. Vì khi mảng được sắp thứ tự rồi thì mới tìm kiếm nhị phân.